实验 4　编译预处理实验

4.1 实验目的

（1）掌握文件包含、宏定义、条件编译和 assert 宏的使用；

（2）练习使用集成开发环境中的调试功能：单步执行、设置断点、观察变量值。

（3）熟悉多文件编译技术

4.2 实验内容

1．程序改错

下面是用宏来计算平方差、交换两数的源程序.在这个源程序中存在若干错误，要求对该程序进行调试修改，使之能够正确完成指定任务。

```
1    /*实验 4-1 改错与跟踪调试题程序：计算平方差、将换两数*/
2    #include<stdio.h>
3    #define SUM a+b
4    #define DIF a-b
5    #define SWAP(a,b)    a=b,b=a
6    int main()
7    {
8            int a,b;
9            printf("Input two integers a, b:");
10             scanf("%d%d", &a,&b);
12           printf("\nSUM=%d\n the difference between square of a and square of b is:%d",SUM, SUM*DIF);
13           SWAP(a,b);
14           printf("\nNow a=%d,b=%d\n",a,b);
15           return 0;
16   }
```

解答：

错误修改：

1. 宏定义 SUM 和 DIF 没有加括号，会导致运算顺序错误，应改为 #define SUM (a + b) 和 #define DIF (a - b)
2. SWAP 宏没有用大括号包裹，可能导致逻辑错误，应使用 { } 包裹宏内容
3. SWAP 宏中未定义临时变量 temp，直接赋值会导致数据丢失，应在宏中添加 int temp = a;
4. SUM * DIF 的写法容易引起歧义，建议加上括号表示为 (SUM) * (DIF) 以提高可读性

2．程序修改替换

下面是用函数实现求三个数中最大数、计算两浮点数之和的程序。在这个源程序中存在若干语法和逻辑错误。

要求：（1）对这个例子程序进行调试修改，使之能够正确完成指定任务；

（2）用带参数的宏替换函数 max，来实现求最大数的功能。

```
/*实验 4-2 程序修改替换题程序*/
#include<stdio.h>
int main(void)
{
    int a, b, c;
    float d, e;
    printf("Input three integers:");
    scanf("%d %d %d",&a,&b,&c);
    printf("\nThe maximum of them is %d\n",max(a,b,c));

    printf("Input two floating point numbers:");
    scanf("%f %f",&d,&e);
    printf("\nThe sum of them is   %f\n",sum(d,e));
    return 0;
}

int max(int x, int y, int z)
```

```
{
    int m=z;
    if (x>y)
        if(x>z) m=x;
    else
        if(y>z) m=y;
     return m;
}

float sum(float x, float y)
{
    return x+y;
}
```

解答：

（1） 源代码清单：

```
#include <stdio.h>
int max(int x, int y, int z);
float sum(float x, float y);
int main(void)
{
    int a, b, c;
    float d, e;
    printf("Input three integers:");
    scanf("%d %d %d", &a, &b, &c);
    printf("\nThe maximum of them is %d\n", max(a, b, c));

    printf("Input two floating point numbers:");
    scanf("%f %f", &d, &e);
    printf("\nThe sum of them is    %f\n", sum(d, e));
    return 0;
```

```
}
int max(int x, int y, int z)
{
    int m = z;
    if (x > y)
    {
        if (x > z)
            m = x;
    }
    else if (y > z)
    {
        m = y;
    }
    return m;
}

float sum(float x, float y)
{
    return x + y;
}
```

（2） 源代码清单：

```
#include <stdio.h>
#define MAX(x, y, z) ((x) > (y) ? ((x) > (z) ? (x) : (z)) : ((y) > (z) ? (y) : (z)))
float sum(float x, float y);
int main(void)
{
    int a, b, c;
    float d, e;
    printf("Input three integers:");
    scanf("%d %d %d", &a, &b, &c);
```

```
        printf("\nThe maximum of them is %d\n", MAX(a, b, c));

        printf("Input two floating point numbers:");
        scanf("%f %f", &d, &e);
        printf("\nThe sum of them is    %f\n", sum(d, e));
        return 0;
    }

    float sum(float x, float y)
    {
        return x + y;
    }
```

3．跟踪调试

下面程序利用 R 计算圆的面积 s，以及面积 s 的整数部分。现要求：

（1）修改程序，使程序编译通过且能运行；

（2）单步执行。进入函数 integerl_fraction 时，watch 窗口中 x 为何值？在返回 main 时，watch 窗口中 i 为何值？

（3）修改程序，使程序能输出面积 s 值的整数部分（要求四舍五入），不会输出错误信息 assertion failed。

```
/*实验 4-3 跟踪调试题程序利用 R 计算圆的面积 s*/
#define    R
int main(void)
{
    float    r, s;
    int s_integer=0;
     printf ("Input a number: ");
     scanf("%f",&r);
     #ifdef    R
         s=3.14159*r*r;
```

```
        printf("Area of round is: %f\n",s);
        s_integer=integer_fraction(s);
        assert((s-s_integer)<0.5);
        printf("The integer fraction of area is %d\n", s_integer);
    #endif
    return 0;
}

int integer_fraction(float x)
{
  int i=x;
  return i;
}
```

解答：

（1） 源代码清单：

```
/*实验 4-3 跟踪调试题程序利用 R 计算圆的面积 s*/
#include <stdio.h>
#include <assert.h>
#define R
int integer_fraction(float x);
int main(void)
{
    float r, s;
    int s_integer = 0;
    printf("Input a number: ");
    scanf("%f", &r);
#ifdef R
    s = 3.14159 * r * r;
    printf("Area of round is: %f\n", s);
    s_integer = integer_fraction(s);
```

```
    assert((s - s_integer) < 0.5);
    printf("The integer fraction of area is %d\n", s_integer);
#endif
    return 0;
}

int integer_fraction(float x)
{
    int i = x + 0.5;
    return i;
}
```

（2）进入函数 integerl_fraction 时，watch 窗口中 x 为 s，在返回 main 时，watch 窗口中 i 为何 x 四舍五入后的整数部分。

（3）同（1）

4．程序设计

（1）三角形的面积是 $area = \sqrt{s(s-a)(s-b)(s-c)}$，其中 $s = (a+b+c)/2$，a, b, c 为三角形的三边，要求编写程序用带参数的宏来计算三角形的面积。定义两个带参数的宏，一个用来求 s，另一个用来求 area。

1）解答思路：算法流程如图 4-1 所示：

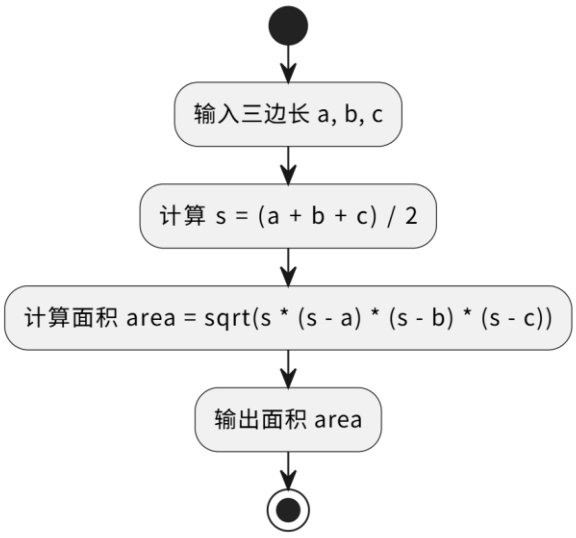

图 4-1 编程题 1 的程序流程图

2）源代码清单：

```
#include <stdio.h>
#include <math.h>
#define S(a, b, c) ((a + b + c) / 2)
#define AREA(a, b, c) (sqrt(S(a, b, c) * (S(a, b, c) - a) * (S(a, b, c) - b) * (S(a, b, c)
- c)))
int main(void)
{
    float a, b, c;
    printf("Input three sides of triangle: ");
    scanf("%f %f %f", &a, &b, &c);
    printf("Area of triangle is: %f\n", AREA(a, b, c));
    return 0;
}
```

3）测试

（a）测试数据：

如表 4-1 所示。

表 4-1 编程题 1 的测试数据

测试用例	程序输入	理论结果	实际结果
用例 1	3 4 5	6.000000	6.000000
用例 2	5 12 13	30.000000	30.000000

（b）对应测试测试用例的运行结果如图 4-2 所示。

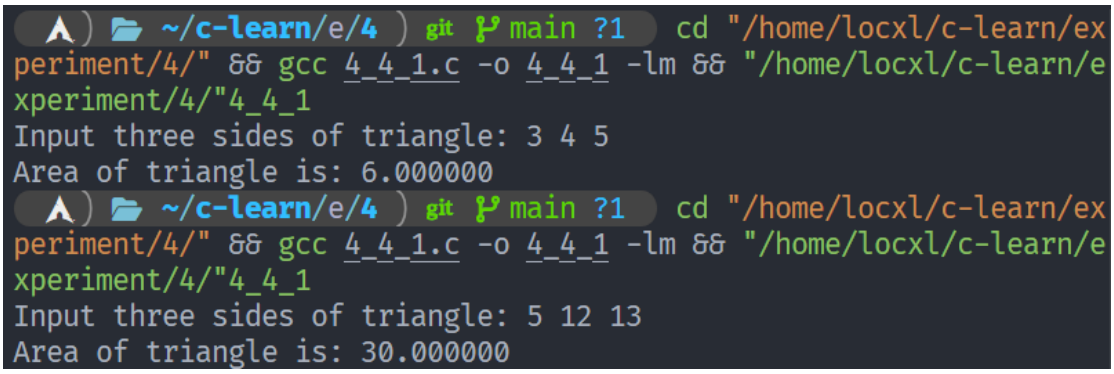

图 4-2 编程题 1 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（2） 用条件编译方法来编写程序。输入一行英文字符序列，可以任选两种方式之一输出：一为原文输出；二为变换字母的大小写后输出。例如小写‘a’变成大写‘A’，大写‘D’变成小写‘d’，其他字符不变。用#define 命令控制是否变换字母的大小写。例如，#define CHANGE 1 则输出变换后的文字，若#define CHANGE 0 则原文输出。

解答：

1） 解题思路：算法流程如图 4-3 所示。

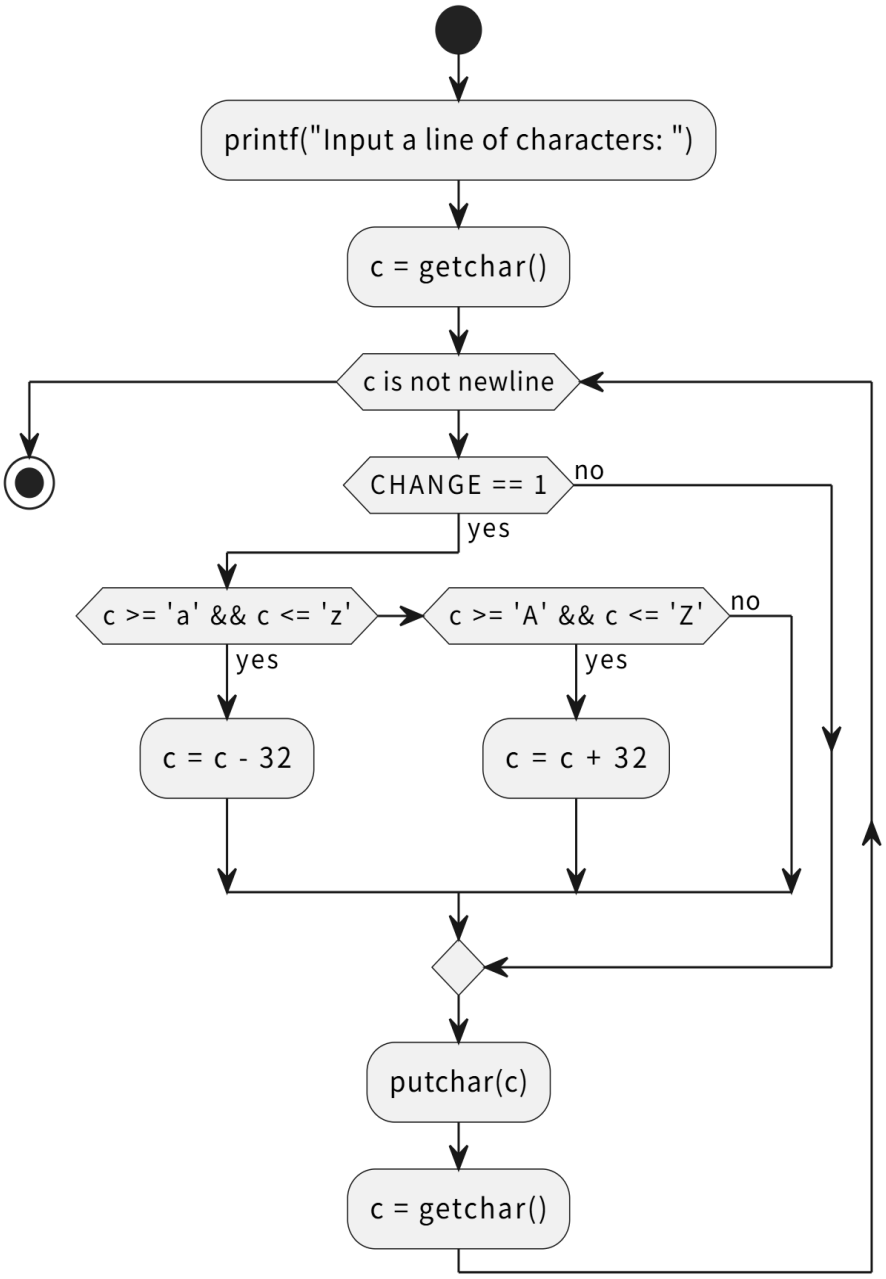

图 4-3 编程题 2 的程序流程图

2） 源代码清单：

```
#include <stdio.h>
#define CHANGE 1
int main(void)
{
    char c;
    printf("Input a line of characters: ");
    while ((c = getchar()) != '\n')
    {
#if CHANGE
        if (c >= 'a' && c <= 'z')
            c -= 32;
        else if (c >= 'A' && c <= 'Z')
            c += 32;
#endif
        putchar(c);
    }
    return 0;
}
```

3） 测试

（a） 测试数据：

如表 4-2 所示

测试用例	程序输入	理论结果	实际结果
用例 1	A	a	a
用例 2	b	B	B

（b） 对应测试测试用例的运行结果如图 4-4 所示。

```
~/c-learn/e/4 ) git main ?1  cd "/home/locxl/c-learn/experi
ment/4/" && gcc 4_4_2.c -o 4_4_2 -lm && "/home/locxl/c-learn/experimen
t/4/"4_4_2
Input a line of characters: A
a%
~/c-learn/e/4 ) git main ?1  cd "/home/locxl/c-learn/experi
ment/4/" && gcc 4_4_2.c -o 4_4_2 -lm && "/home/locxl/c-learn/experimen
t/4/"4_4_2
Input a line of characters: b
b%
```

图 4-4 编程题 2 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（3）假设一个 C 程序由 file1.c 和 file2.c 两个源文件及一个 file.h 头文件组成，file1.c、file2.c 和 file.h 的内容分别如下所述。试编辑该多文件 C 程序，补充 file.h 头文件内容，然后编译和链接。然后运行最后生成的可执行文件。

```
/*源文件 file1.c 的内容*/
#include "file.h"
int x,y;                    /* 外部变量的定义性说明 */
char ch;                    /* 外部变量的定义性说明 */
int main(void)
{
    x=10;
    y=20;
    ch=getchar();
    printf("in file1 x=%d,y=%d,ch is %c\n",x,y,ch);
    func1();
    return 0;
}

/*源文件 file2.c 的内容为：*/
#include "file.h"
void func1(void)
{
    x++;
    y++;
    ch++;
    printf("in file2 x=%d,y=%d,ch is %c\n",x,y,ch);
```

}

解答：

（1）file.h 源代码：

```
#ifndef FILE_H
#define FILE_H

#include <stdio.h>

extern int x, y;
extern char ch;

void func1(void);

#endif
```

（2）编译指令：gcc file1.c file2.c -o app

（3）测试;

（a）测试数据：

如表 4-3 所示

表 4-3 编程题 3 的测试数据

测试用例	程序输入	理论结果	实际结果
用例 1	B	in file1 x=10,y=20,ch is B in file2 x=11,y=21,ch is C	in file1 x=10,y=20,ch is B in file2 x=11,y=21,ch is C
用例 2	a	in file1 x=10,y=20,ch is a in file2 x=11,y=21,ch is b	in file1 x=10,y=20,ch is a in file2 x=11,y=21,ch is b

（b）对应测试测试用例的运行结果如图 4-4 所示。

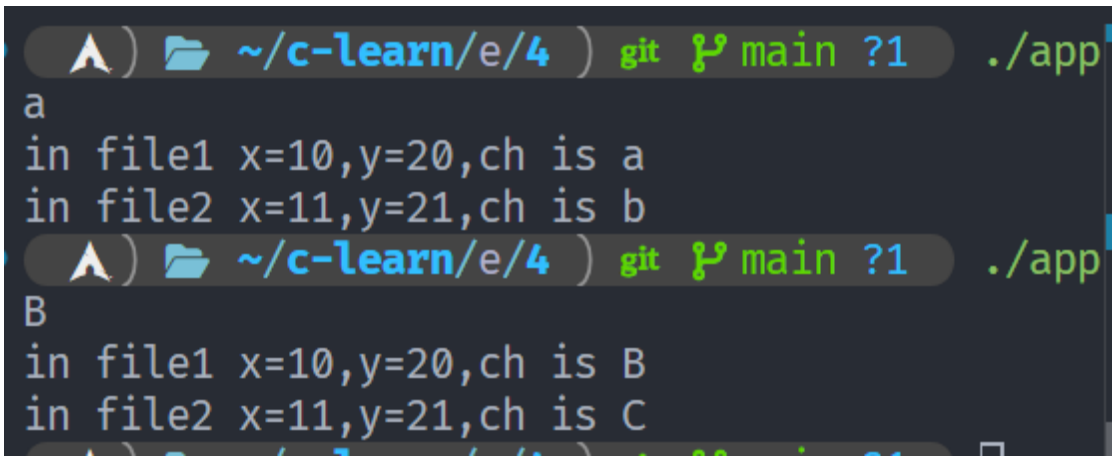

图 4-5 编程题 3 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

4.3 实验小结

在本次实验过程中，遇到了多个问题，如宏定义未加括号导致运算顺序错误、SWAP 宏缺少临时变量引发数据丢失、函数未声明导致调用错误、assert 断言失败等。针对这些问题，通过仔细分析代码逻辑，结合调试工具观察变量变化，逐步排查并修改了错误。例如，在宏定义中添加括号以保证运算顺序，为 SWAP 宏引入临时变量，补全函数声明，以及调整 assert 条件避免浮点数精度问题。通过本次实验，深刻体会到宏定义和条件编译的灵活性与潜在风险，认识到合理使用调试工具对程序分析的重要性，同时也增强了对多文件编译结构的理解与应用能力。